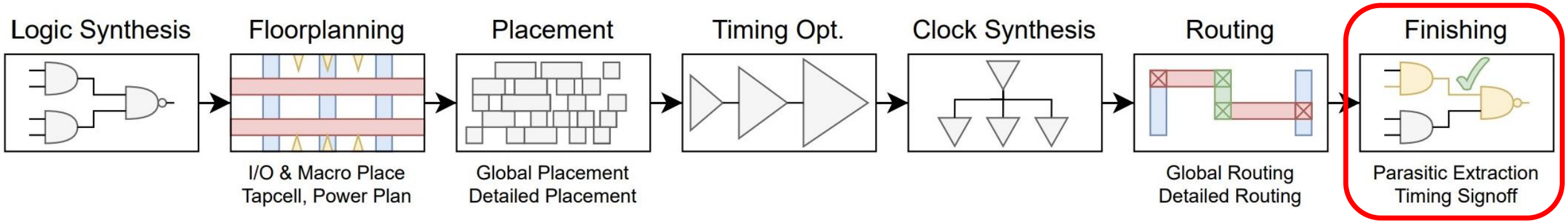
Finishing


Logic Synthesis
Floorplanning
Placement
Timing Opt.
Clock Synthesis
Routing
Finishing
I/O & Macro Place
Tapcell, Power Plan
Global Placement
Detailed Placement
Global Routing
Detailed Routing
Parasitic Extraction
Timing Signoff

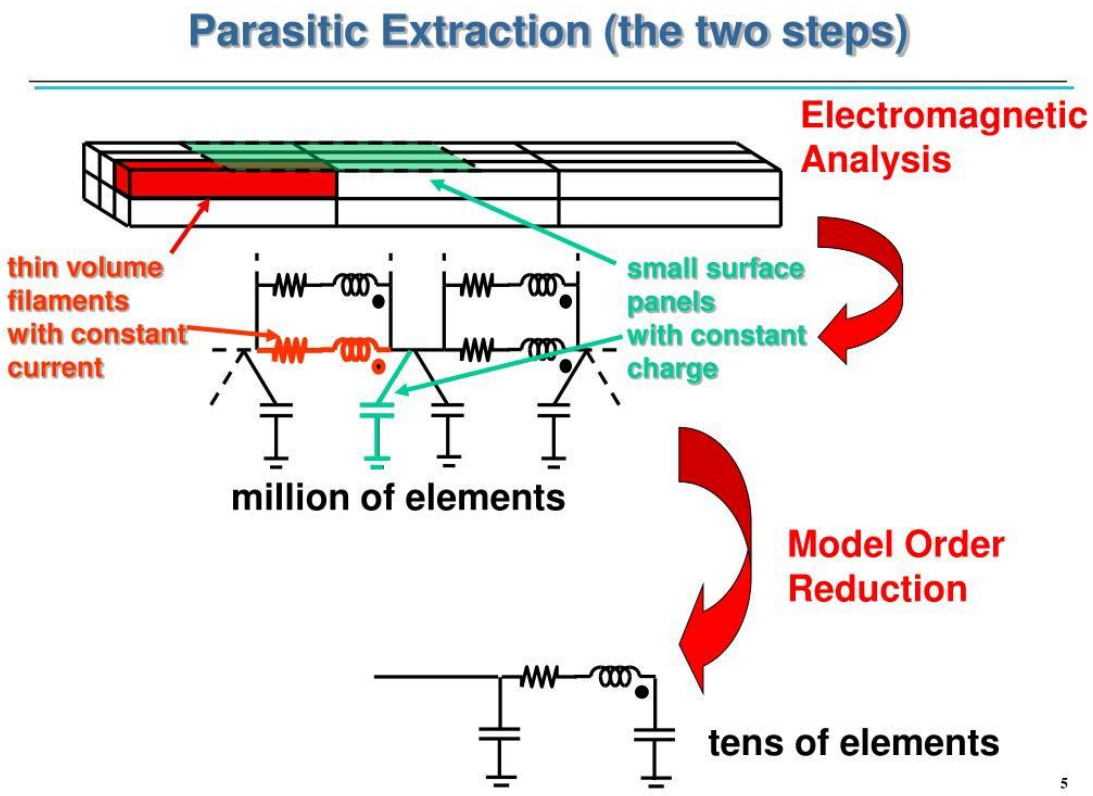
Finishing

- Until now, using high quality approximations of timing and parasitics
- Need “signoff quality” guarantees
- Perform a full 3D parasitic extraction (PEX) and use high quality models to validate timing (STA)
  - Remember: higher quality = more accurate but longer runtime!
- If violations are found, try a few quick optimizations to fix
  - “Engineering Change Order” (ECO)
- Prepare design for export (filler insertion and GDS merge)

Effect of Capacitance (Reduced [illegible] (mostly wires).

Parasitic Extraction

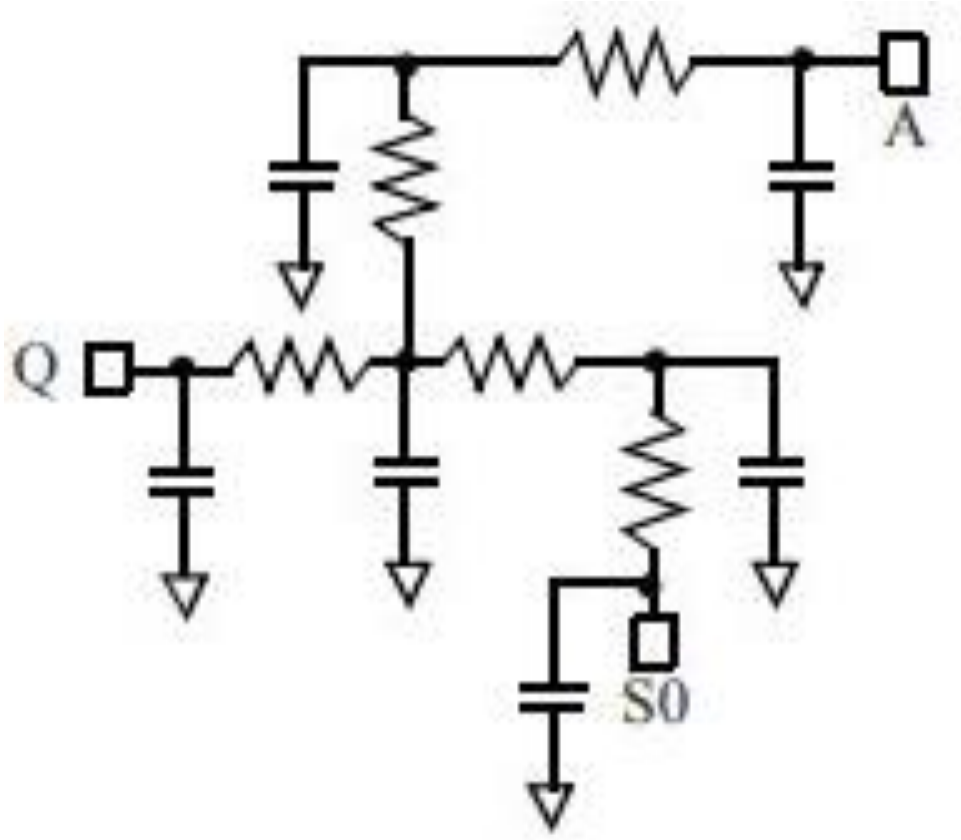
- 3D parasitic extraction on R and C (usually not L for digital)
- Perform meshing on wires and model system using Maxwell's equations
- Perform reductions on the system to simplify
- Some tools pre-characterize patterns and simply perform a lookup in a table



PEX → STA → ECO → Filler → GDS

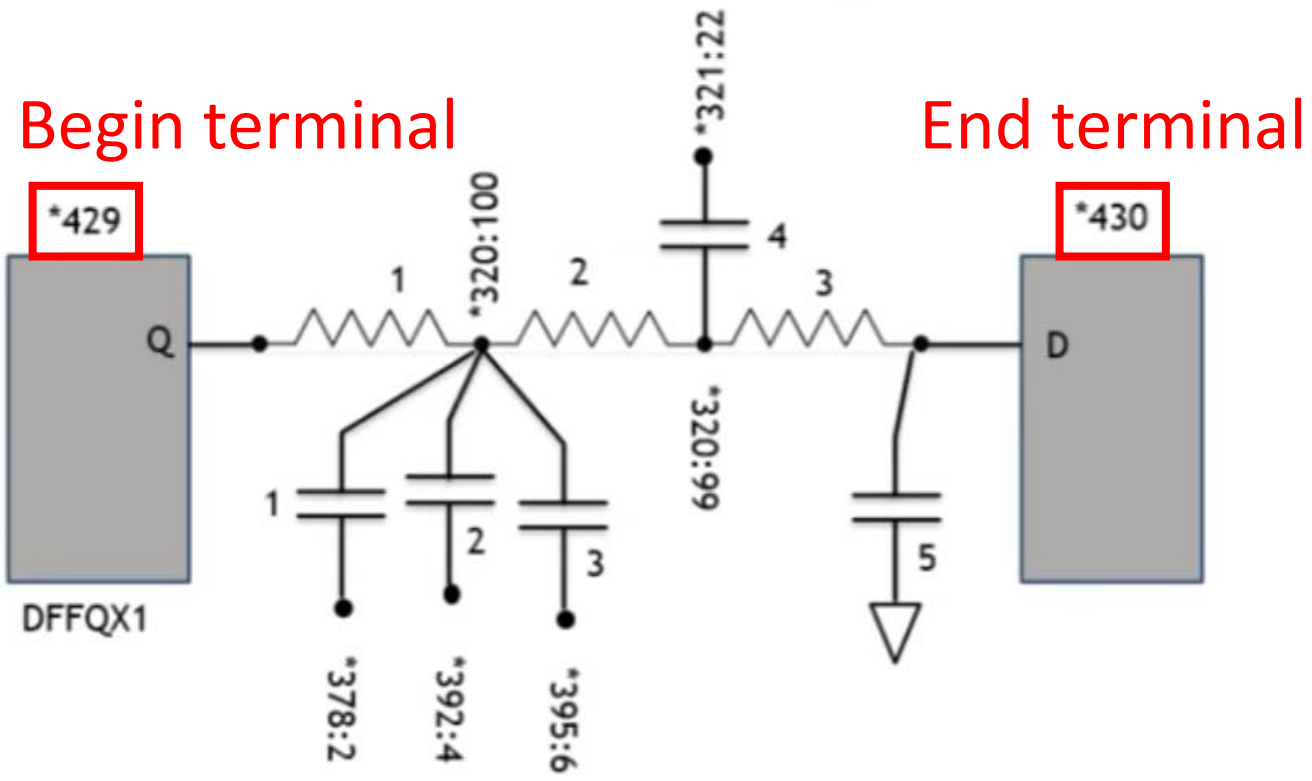
Parasitic Extraction

- Output is usually a **SPEF** file
  - Standard Parasitic Exchange Format
  - Or, a tool may pass the info internally
- SPEF can use a few different models, but most common is **distributed net**
  - Each net is divided into segments
  - Each segments has its own RC values



PEX → STA → ECO → Filler → GDS

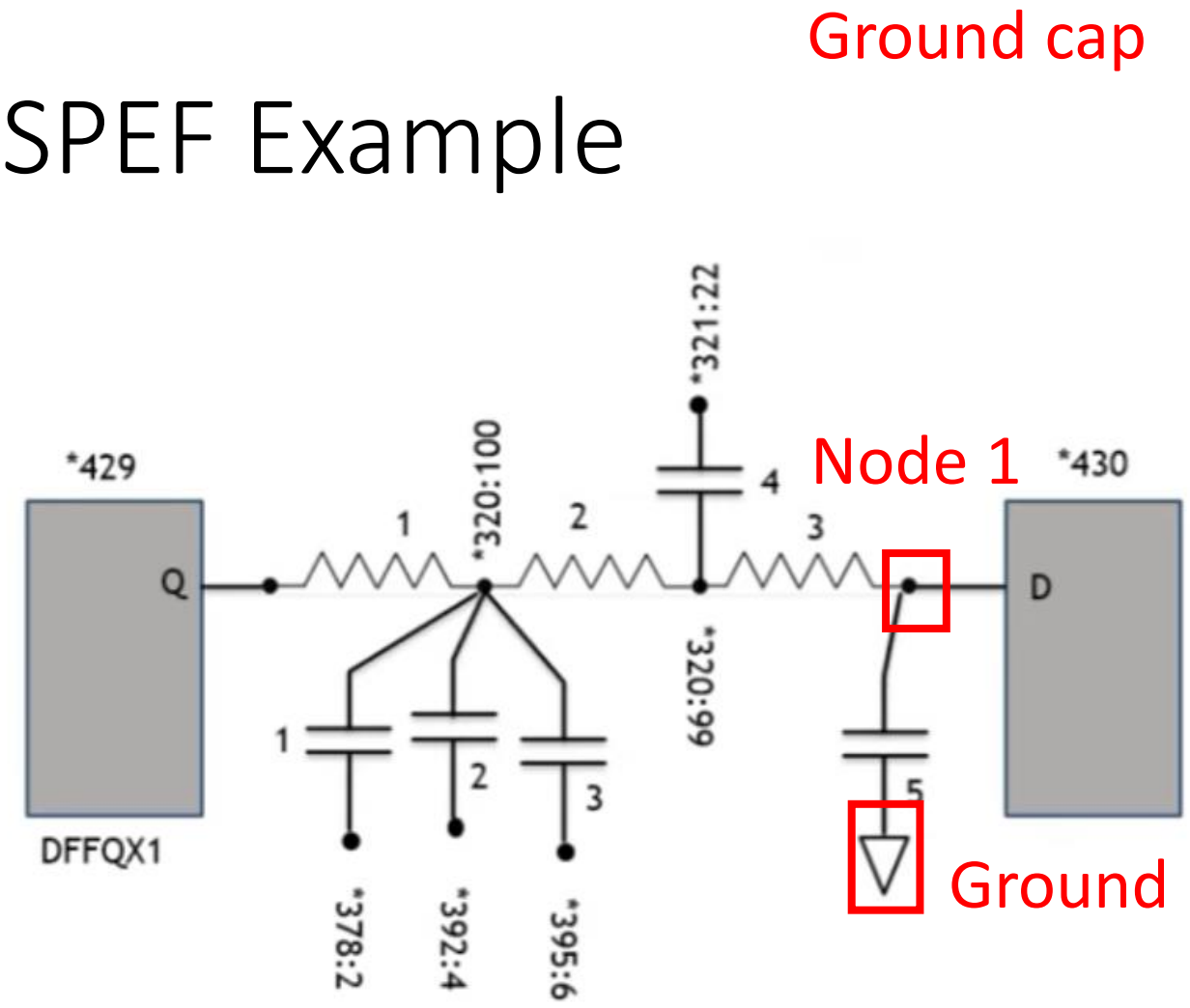
SPEF Example




```
*D_NET *320 0.183506

*CONN
*I *430:D I  *C 5.27 27.425
*I *429:Q O  *C 4.27 27.425 *D DFFQX1

*CAP
1 *320:100 *378:2 0.083118
2 *320:100 *392:4 0.028394
3 *320:100 *395:6 0.017284
4 *320:99   *321:22 0.00237
5 *430:D 0.05234

*RES
1 *429:Q *320:100 0.3400
2 *320:100 *320:99  0.9162
3  *320:99 *430:D 0.3400

*END
```

PEX → STA → ECO → Filler → GDS

SPEF Example



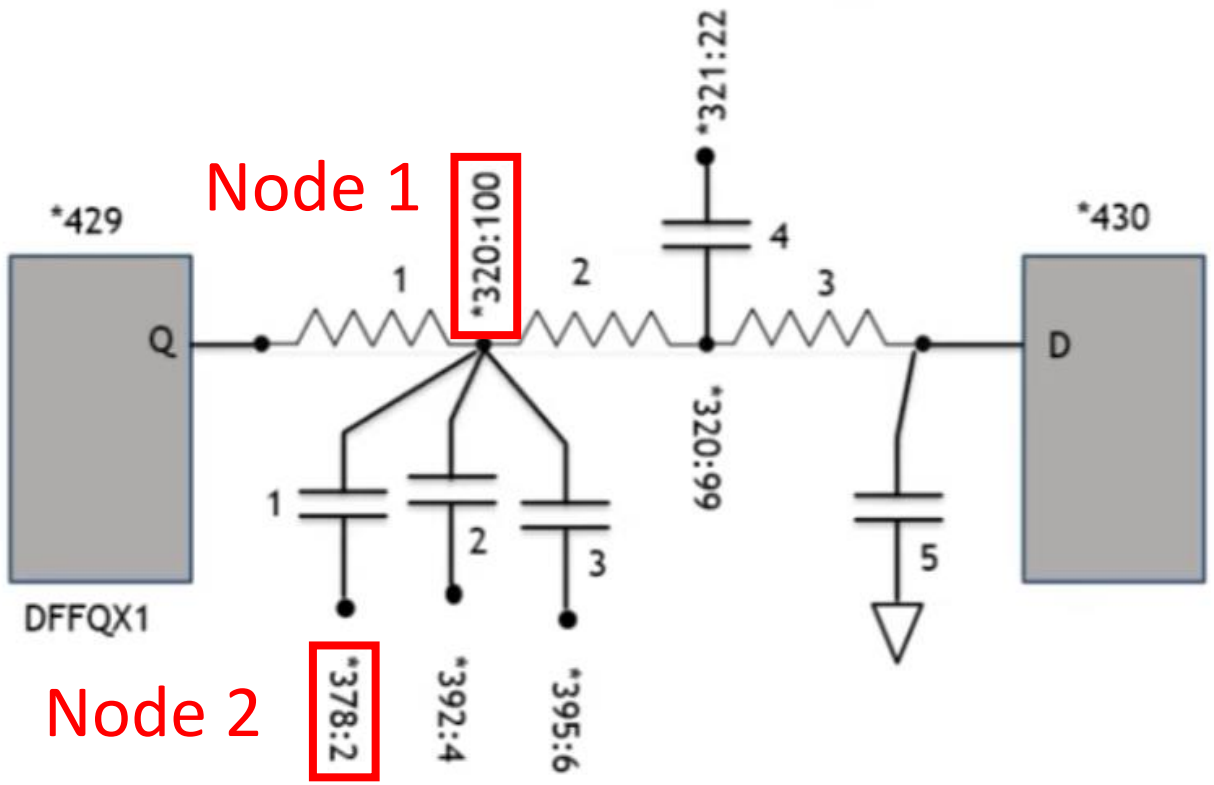

```
*D_NET *320 0.183506

*CONN
*I *430:D I  *C 5.27 27.425
*I *429:Q O  *C 4.27 27.425 *D DFFQX1

*CAP
1 *320:100 *378:2 0.083118
2 *320:100 *392:4 0.028394
3 *320:100 *395:6 0.017284
4 *320:99   *321:22 0.00237
5 *430:D 0.05234

*RES
1 *429:Q *320:100 0.3400
2 *320:100 *320:99  0.9162
3  *320:99 *430:D 0.3400

*END
```

Capacitance value

PEX → STA → ECO → Filler → GDS

SPEF Example

Coupling cap between nodes 320 and 378



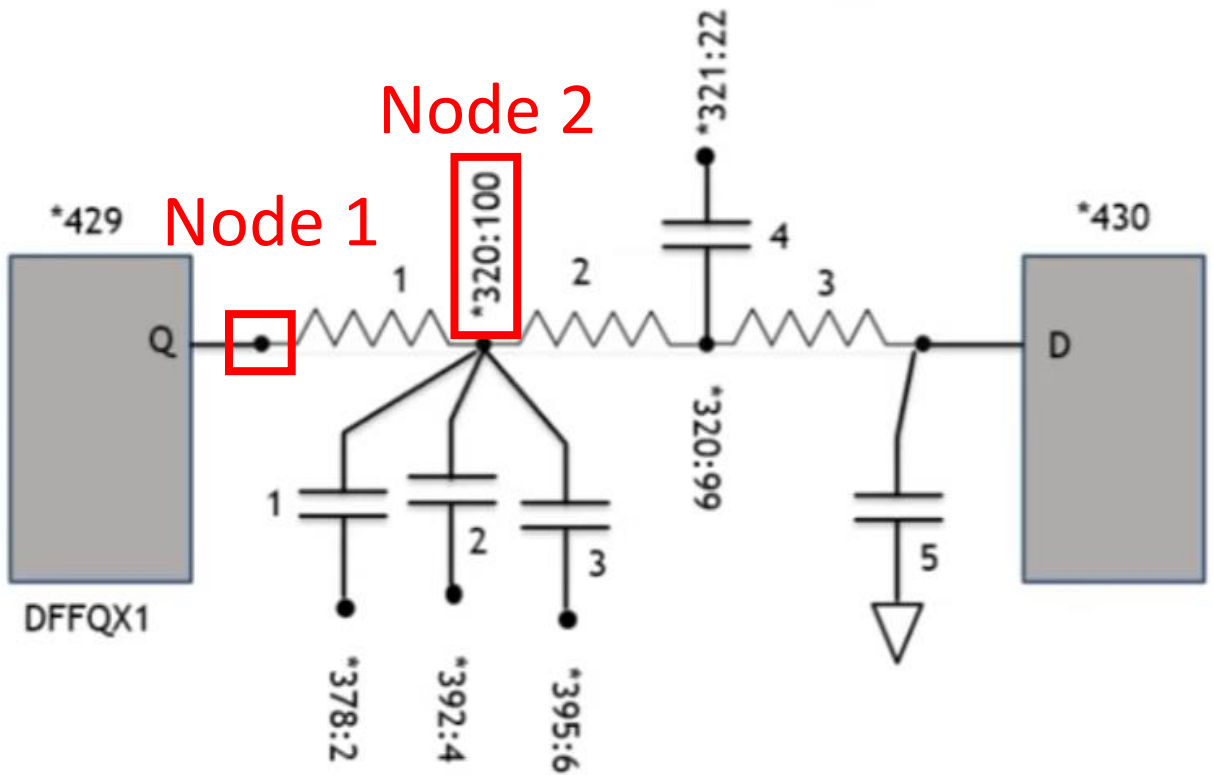

```
*D_NET *320 0.183506

*CONN
*I *430:D I  *C 5.27 27.425
*I *429:Q O  *C 4.27 27.425 *D DFFQX1

*CAP
1 *320:100 *378:2 0.083118
2 *320:100 *392:4 0.028394
3 *320:100 *395:6 0.017284
4 *320:99  *321:22 0.00237
5 *430:D 0.05234

*RES
1 *429:Q *320:100 0.3400
2 *320:100 *320:99  0.9162
3  *320:99 *430:D 0.3400

*END
```

Capacitance value

PEX → STA → ECO → Filler → GDS

SPEF Example

Resistance between nodes 429 and 320



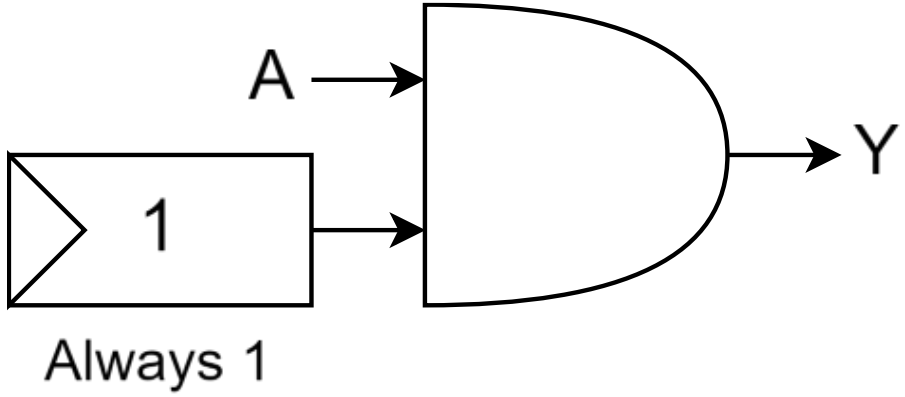

```
*D_NET *320 0.183506

*CONN
*I *430:D I  *C 5.27 27.425
*I *429:Q O  *C 4.27 27.425 *D DFFQX1

*CAP
1 *320:100 *378:2 0.083118
2 *320:100 *392:4 0.028394
3 *320:100 *395:6 0.017284
4 *320:99   *321:22 0.00237
5 *430:D 0.05234

*RES
1 *429:Q *320:100 0.3400
2 *320:100 *320:99  0.9162
3  *320:99 *430:D 0.3400

*END
```

Resistance value

PEX → STA → ECO → Filler → GDS

Static Timing Analysis (STA)

- **Dynamic**: circuit simulation. Enumerate all different states and possible transitions of the circuit
  - Infeasible for large circuits – combinatorial explosion of possible states
- **Static**: pessimistic assumption that worst transition is always possible
  - Remove dependency on state. Analysis is vastly simplified
- Example:
  - Dynamic analysis possible transitions (A,B):
    - (0->1,1), (1->0,1)
  - Static analysis possible transitions (A,B):
    - (0->1,0), (0->1,1), (1->0,0), (1->0,1), …
    - (0->1, 0->1), (0->1, 1->0), (1->0, 1->0), …



Static Timing Analysis (STA)

- Using the worst-case transitions (defined in Liberty model), parasitic capacitances (defined in SPEF or internal to tool), analyze the netlist
- Represent netlist as Directed Acyclic Graph (DAG)
- Longest path algorithm to find arrival time
- Repeat process for both setup and hold

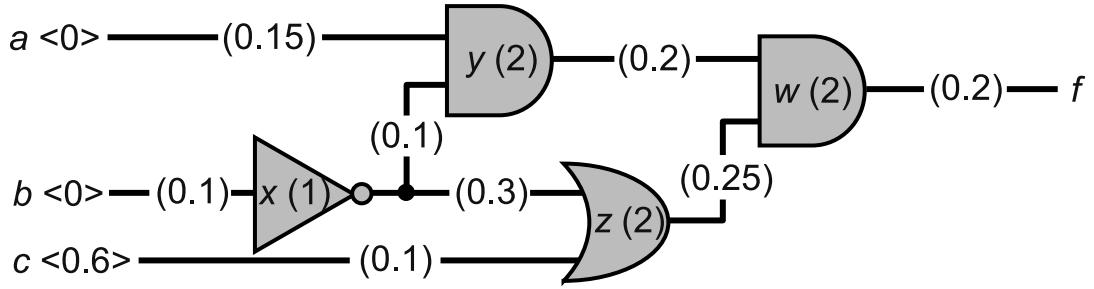


Fig. 8.2 Three inputs *a*, *b* and *c* are annotated with times at which signal transitions occur in angular brackets. Each edge and gate is annotated with its delay in parentheses.

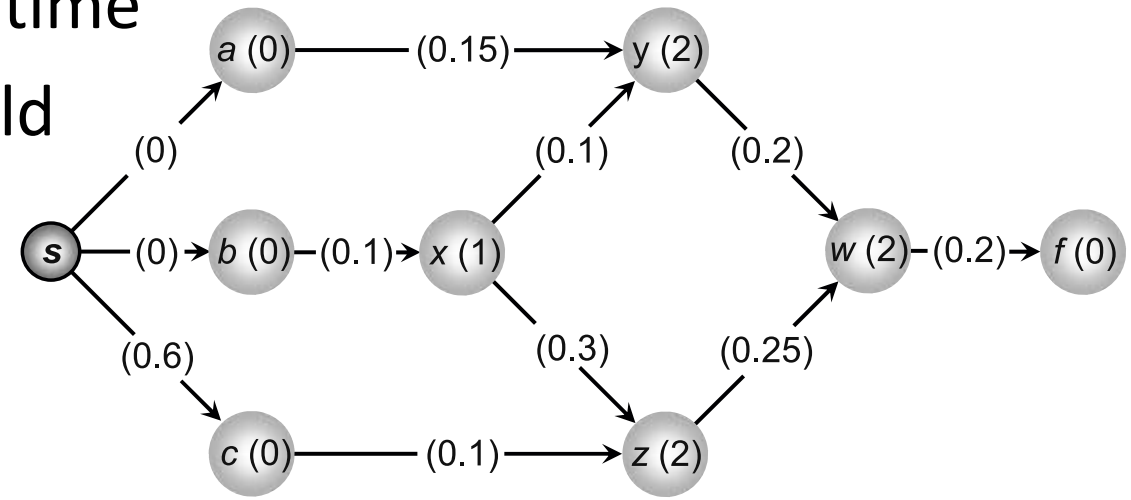


Fig. 8.3 DAG representation using gate node convention of the circuit in Fig. 8.2.

Engineering Change Order (ECO)

- Older engineering / chip design term, but still used
- Generally refers to “inserting a logic change directly into the netlist after it has already been processed by an automatic tool”
- In practice: insert very small changes into the design to correct errors
- Two main kinds of ECO:
  - Pre-mask ECO (**common**): Change design before mask fabrication (tapeout)
    - Fix small errors in timing discovered using highest accuracy models
  - Post-mask ECO (**rare**): Change the design after mask fabrication (tapeout)
    - Fix design errors, usually by minimizing number of masks (layers) that must be remade

Pre-mask ECO

- What's the difference between ECO and normal optimization?
  - Iteration time is very expensive here
  - Everything is simulated and calculated with highest-quality models
- Common techniques
  - **In-place cell swapping**: swap in cells with the same *footprint* to avoid rerouting, redoing PEX. Ex: Vt swapping, channel-length swapping
  - Buffer insertion, gate resizing: same techniques as before, but now the calculations are more expensive

Pre-mask ECO

- Can loop this process as many times as the designer wants
- Generally, the benefit will plateau after a few iterations
  - If it doesn't converge, designer must make changes at an earlier design stage
- Once timing is closed, the designing is done!!!
- But, several other verification steps may remain

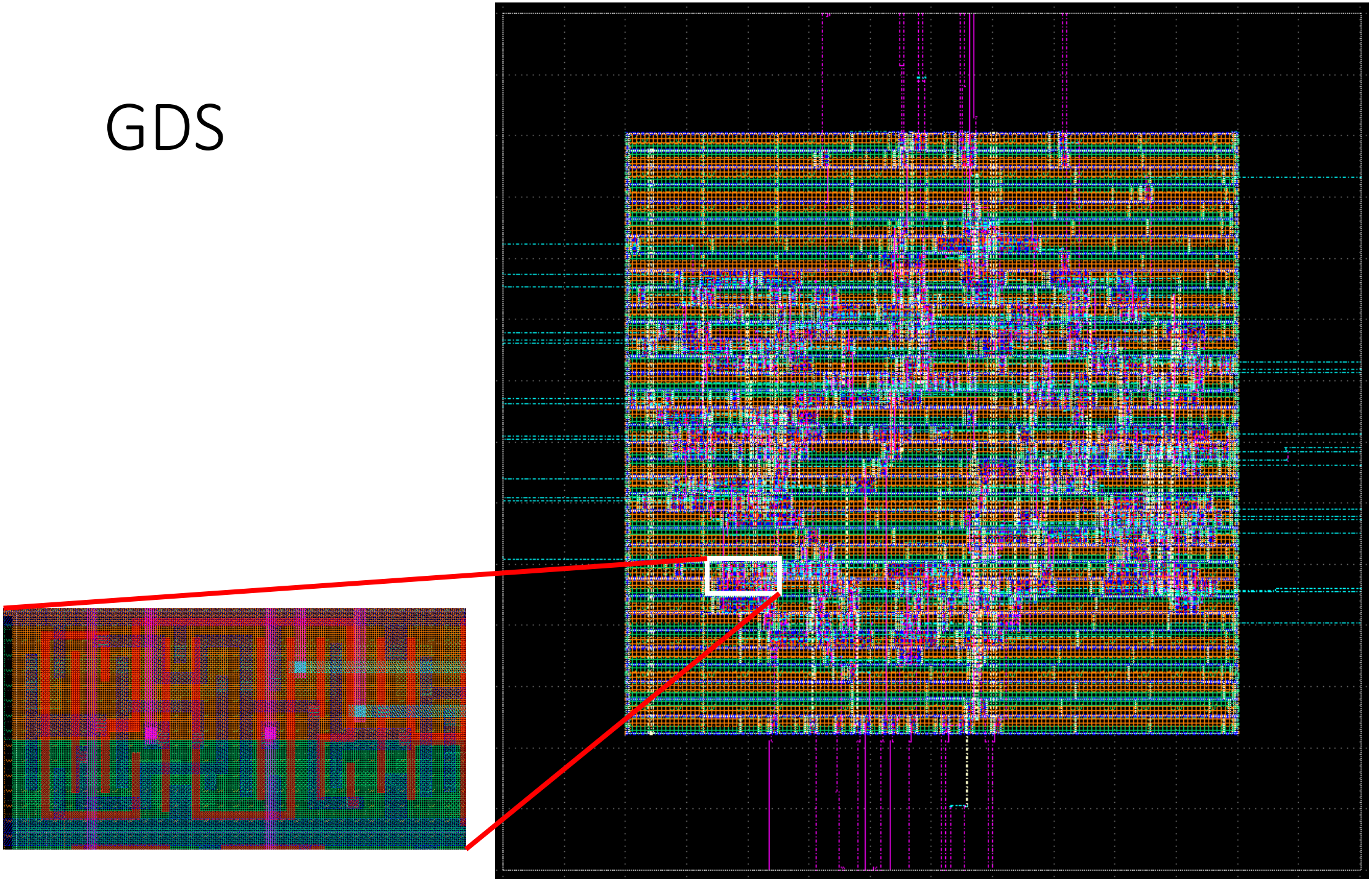
Filler Cell Insertion

- Cell rows often still have unfilled cell sites (<100% utilization)
- These breaks can leave gaps in the nwell, implantation layers, etc.
- We're done adding gates, we can now fill in these gaps with **filler cells**
- Filler cells usually have nwell, power rails, sometimes dummy transistors to help with density rules
- Some flows may add filler cells at an earlier stage and remove them when adding gates. But they aren't truly needed until now

GDS Merge

- Design has been using abstract physical views (LEF) this whole time
- Merge in the detailed views with transistor layers, etc.

GDS



Acknowledgement

- Yosys Verilog and AST Frontends. https://blog.eowyn.net/yosys/CHAPTER_Verilog.html
- AUCOHL/DFFRAM. https://github.com/AUCOHL/DFFRAM
- A. B. Kahng, et al. *RTL-MP: Toward Practical, Human-Quality Chip Planning and Macro Placement*. ISPD 2022.
- S. K. Moore. *Ending an Ugly Chapter in Chip Design: Study tries to settle a bitter disagreement over Google's chip design.* IEEE Spectrum, 04 April 2023.
- A. Goldie and A. Mirhoseini. *Statement on Reinforcement Learning for Chip Design*. 24 March 2023.